

Capitolo 1

Introduzione a Paradigmi e Linguaggi di Programmazione

1.1 Definizioni di Base per Paradigmi e Linguaggi

- Un linguaggio di programmazione è uno strumento linguistico attraverso il quale è possibile formalizzare un algoritmo in modo tale che esso sia eseguibile da un computer in quanto macchina programmabile. Il risultato di questa formalizzazione prende il nome di programma informatico o applicazione informatica ed è costituito da una collezione di istruzioni memorizzate in un file detto file sorgente.
- Ogni linguaggio di programmazione ha le proprie regole lessicali e grammaticali. Le prime specificano come combinare i simboli per ottenere le parole o lessemi del linguaggio, mentre le seconde descrivono come combinare le parole per formare prima i sintagmi e poi le frasi sintatticamente ben formate. I linguaggi di programmazione hanno lessico e sintassi molto più semplici di quelli dei linguaggi naturali e sono sostanzialmente classificabili come *linguaggi liberi dal contesto* nella gerarchia di Chomsky.
- Ogni linguaggio di programmazione è inoltre dotato di una *semantica*, che *attribuisce un significato a ogni costrutto linguistico*, e di conseguenza a ogni programma, in modo tale da stabilirne l'effetto a *tempo d'esecuzione*. Non tutte le frasi sintatticamente ben formate del linguaggio hanno un significato.
- Esempio: la frase “il gatto guida una motocicletta” è costituita dai lessemi “il”, “gatto”, “guida”, “una”, “motocicletta” e dai sintagmi “il gatto” (soggetto), “guida” (predicato), “una motocicletta” (complemento), quindi è sintatticamente ben formata in linguaggio naturale, ma non ha senso nella realtà.
- Ciascun computer è dotato di due linguaggi di programmazione che sono specifici del proprio processore: *il linguaggio macchina e il linguaggio assembly*. Ogni istruzione di questi linguaggi è costituita dal codice operativo dell'istruzione (addizione, sottrazione, moltiplicazione, divisione, trasferimento dati da/verso memoria o salto condizionato/incondizionato di istruzioni) e dai valori o dagli indirizzi di memoria degli operandi.
- Nel linguaggio macchina codici e indirizzi sono espressi in binario, mentre nel linguaggio assembly possono essere espressi in formato mnemonico tramite identificatori. Di conseguenza le istruzioni in linguaggio macchina sono direttamente eseguibili dal computer, mentre quelle in linguaggio assembly – che sono più intelligibili – necessitano di essere preventivamente trasformate nelle corrispondenti istruzioni in linguaggio macchina tramite un traduttore detto assembler.
- I programmi scritti in linguaggio macchina o assembly non sono portabili, cioè non possono essere eseguiti su computer aventi un processore diverso. Ancor peggio, questi linguaggi non consentono di scrivere programmi agevolmente perché si collocano a un livello di astrazione troppo basso per la mente umana.

- Dalla metà degli anni 1950 sono stati sviluppati linguaggi di programmazione di alto livello di astrazione in cui è più facile scrivere programmi perché, rispetto ai linguaggi macchina e assembleativi, essi sono più vicini al linguaggio naturale delle persone e alla notazione matematica di uso comune. Un'importante conseguenza è la naturale portabilità dei programmi su computer diversi, conducendo pertanto a compimento una completa separazione tra hardware e software iniziata con l'idea di computer a programma memorizzato di Von Neumann e teorizzata ancor prima da Turing.
- Un programma scritto in un linguaggio di alto livello di astrazione non è però direttamente eseguibile da un computer, in quanto necessita di essere preventivamente tradotto in un programma equivalente scritto nel linguaggio macchina o assembleativo di quel computer. Dato il diverso livello di astrazione, ogni istruzione del programma originario verrà tradotta in una sequenza di più istruzioni macchina o assembleative. Il traduttore è detto **compilatore** o **interprete** a seconda che la traduzione venga effettuata una volta per tutte prima dell'esecuzione del programma (nel qual caso il risultato della traduzione viene memorizzato in un file detto file eseguibile) o contestualmente all'esecuzione istruzione per istruzione.
- I numerosi linguaggi di programmazione di alto livello di astrazione che sono stati ideati possono essere molto diversi tra loro, talché hanno dato luogo a più paradigmi di programmazione. Un paradigma di programmazione è un insieme di concetti, costrutti e astrazioni che determinano il processo di stesura di un programma in certi linguaggi. Ad un estremo abbiamo i linguaggi di programmazione che si concentrano solo sulla definizione di cosa calcolare senza specificarne il come, esprimendo soltanto la conoscenza relativa al dominio del problema da risolvere. All'estremo opposto abbiamo i linguaggi di programmazione in cui occorre indicare pure tutti i dettagli su come calcolare la soluzione del problema.
- Esempio: il fattoriale di $n \geq 1$ è n moltiplicato per il fattoriale di $n - 1$ con fattoriale di 1 uguale a 1 vs. per calcolare il fattoriale di n bisogna moltiplicare 1 per 2 poi il risultato va moltiplicato per 3 poi ancora il risultato va moltiplicato per 4 e così via fino ad arrivare alla moltiplicazione per n .

1.2 Programmazione Imperativa: Procedurale e a Oggetti

- Il paradigma imperativo trae la sua origine dai linguaggi macchina e assembleativi e ricomprende sia la programmazione procedurale che la programmazione a oggetti.
- Nel paradigma imperativo i programmi sono sequenze di istruzioni che prescrivono dettagliatamente quali operazioni effettuare e in quale ordine. L'istruzione fondamentale è l'istruzione di assegnamento, la quale stabilisce come modificare il contenuto di una locazione di memoria individuata da un identificatore di variabile. Per il controllo del flusso di esecuzione sono disponibili istruzioni di selezione e di ripetizione che, grazie al teorema fondamentale della programmazione strutturata di Böhm e Jacopini, evitano il ricorso a istruzioni di salto condizionato/incondizionato. È altresì supportata la ricorsione. Sono presenti tipi di dati scalari per valori numerici, logici, enumerati, caratteri e indirizzi, tipi di dati strutturati omogenei quali array, stringhe e insiemi ed eterogenei quali record e file, nonché tipi di dati definiti dal programmatore. Le rispettive variabili sono allocabili staticamente o dinamicamente.
- Nella programmazione procedurale le istruzioni di un programma sono raggruppate in blocchi detti procedure per evitare ridondanze di codice. Ogni procedura è definita una sola volta nel programma e può essere poi invocata da più istruzioni del programma, presenti in altre procedure o nella procedura stessa. La definizione della procedura è basata su dati di valore ignoto chiamati parametri formali che vengono inizializzati al momento dell'invocazione della procedura attraverso i corrispondenti parametri effettivi, i quali possono essere passati per valore o per indirizzo. La procedura invocata può restituire un risultato da utilizzare nell'istruzione contenente l'invocazione.
- Esempi di linguaggi di programmazione procedurali:
 - **Fortran**: 1957, John Backus presso IBM, applicazioni scientifiche, inizialmente privo di articolazione in sottoprogrammi e supporto a ricorsione e allocazione dinamica.
 - **Cobol**: 1960, comitato industriale e governativo americano tra cui Jean Sammet, applicazioni gestionali, sintassi molto verbosa, limitazioni iniziali simili a quelle del Fortran.

- **Algol**: 1958, comitato accademico europeo e americano, descrizione chiara degli algoritmi, sintassi in formato Backus-Naur, blocchi di istruzioni delimitati da inizio e fine, privo di istruzioni di input/output predefinite.
 - **PL/I**: 1966, IBM, misto di Fortran e Cobol influenzato da Algol da usare come unico linguaggio per applicazioni scientifiche e gestionali oltre che sistemiche su IBM 360 quale unica architettura che soppiantava tutte le precedenti, privo di parole riservate.
 - **Basic**: 1964, Kemeny e Kurtz presso Dartmouth College, programmazione per principianti, interpretato anziché compilato, inizialmente privo di supporto per la programmazione strutturata (famosa critica di Dijkstra), evolutosi in Visual Basic nel 1991 presso Microsoft.
 - **Pascal**: 1970, Niklaus Wirth presso ETH Zurigo, insegnamento della programmazione strutturata, evoluzione di Algol in cui i blocchi vengono specializzati in procedure e funzioni eventualmente annidate, tipi ricorsivi e dinamici (per liste, alberi, grafi) aggiunti a tipi scalari e array.
 - **C**: 1972, Dennis Ritchie presso Bell Labs, evoluzione di B – derivato da Ken Thompson come semplificazione di BCPL, progettato nel 1967 da Martin Richards presso l'Università di Cambridge – messa a punto per consentire agli sviluppatori del sistema operativo Unix di implementare funzionalità, no annidamento di funzioni, portabilità coniugata con vicinanza alla macchina.
 - **Modula**: 1975, Niklaus Wirth presso ETH Zurigo, evoluzione di Pascal basata su moduli come unità di programmazione compilabili separatamente che sono comprensive di dati e procedure – estensione di unità Pascal e librerie C – e su processi e segnali per la programmazione concorrente.
 - **Ada**: 1980, Jean David Ichbiah presso Honeywell Bull, safety-critical embedded software tipico dell'industria aerospaziale, sistema di tipi molto evoluto, supporto a modularità basato su package e a programmazione concorrente basato su task e scambio sincrono di messaggi.
- La programmazione a oggetti è un'evoluzione della programmazione procedurale finalizzata ad aumentare l'efficienza del processo di produzione e manutenzione del software, attraverso un innalzamento del livello di astrazione ottenuto separando i dati dalle procedure pur collocandoli vicino a queste ultime.
 - Nella programmazione a oggetti le istruzioni vengono raggruppate assieme ai loro dati, dando luogo alle cosiddette classi. Una classe è una collezione di attributi (dati) e metodi (procedure), collettivamente chiamati membri, la cui visibilità può essere pubblica, protetta o privata a seconda che siano accessibili da tutte le altre classi, dalla classe e relative estensioni oppure solo dalla classe stessa. Ispirandosi al concetto di tipo di dato astratto, che comprende sia un insieme di valori che le operazioni su di esso, ciascuna classe esibisce un'interfaccia costituita dalla dichiarazione dei soli membri pubblici e incapsula la dichiarazione di tutti gli attributi e la definizione di tutti i metodi. Grazie al meccanismo dell'ereditarietà è possibile definire gerarchie di classi, promuovendo il riutilizzo del codice attraverso l'estensione di classi già esistenti, nonché supportare il polimorfismo, mediante classi astratte che lasciano alle classi derivate il compito di completare la definizione dei suoi metodi.
 - Un programma a oggetti è una collezione di oggetti che interagiscono tra loro invocando reciprocamente i relativi metodi per consultare o modificare i rispettivi dati e sono dotati di meccanismi per gestire eccezioni quando si verificano determinate situazioni. Un oggetto è un'istanza di una classe avente valori specifici ed eventualmente immodificabili per gli attributi della classe, quindi è assimilabile all'allocazione dinamica con relativa inizializzazione di una variabile avente come tipo quella classe. Gli oggetti di una classe hanno gli stessi metodi, ma possono differire per i valori degli attributi.
 - Esempi di linguaggi di programmazione a oggetti:
 - **Simula**: 1965, Dahl e Nygaard presso Norwegian Computing Center, programmi di simulazione al computer, coroutine come variante di subroutine (cioè procedura) la cui esecuzione può essere sospesa e poi ripresa con istruzioni apposite, classi e sottoclassi di oggetti.
 - **Smalltalk**: 1972, Alan Kay presso Learning Research Group di Xerox, apprendimento costruzionista, programmi compatti, tutti i valori sono oggetti (anche numeri/booleani/caratteri), scambio di messaggi (anche per il controllo del flusso), gestione di eccezioni, compilato in bytecode che viene poi interpretato da una macchina virtuale, software design pattern, interfacce grafiche, ambiente di sviluppo integrato, riflessione come capacità di un programma di ispezionare e modificare struttura e comportamento a tempo d'esecuzione.

- **C++**: 1985, Bjarne Stroustrup presso Bell Labs, evoluzione di C orientata agli oggetti, template per consentire a funzioni e classi di operare su tipi generici specificati come parametri, ammesso il riuso di operatori esistenti in nuovi tipi di dati definiti dal programmatore, ereditarietà anche da molteplici classi base, distruttori di oggetti, try-catch-throw per la gestione di eccezioni, spazi di nomi per evitare conflitti tra nomi, poi esteso con thread per la programmazione concorrente.
- **Eiffel**: 1986, Bertrand Meyer, affidabilità del software, design by contract in cui asserzioni come precondizioni e postcondizioni oppure invarianti di classe o di ciclo assicurano la correttezza di un programma, gestione delle eccezioni basata su tali asserzioni, allocazione dinamica della memoria comprensiva di garbage collection automatica.
- **Java**: 1996, James Gosling presso Sun Microsystems, verso applicazioni web tramite applet e servlet, compilato in bytecode che viene poi interpretato o compilato a tempo d'esecuzione dalla Java Virtual Machine, edizioni diverse per ambiti di applicazione diversi, classi organizzate in package, tutti i valori sono oggetti eccetto numeri/booleani/caratteri (per motivi prestazionali), non supportati il riuso di operatori e l'ereditarietà multipla (per chiarezza), privo degli operatori di basso livello di C/C++ e del tipo puntatore, diverse forme di garbage collection automatica, supporto alla programmazione concorrente basato su thread.
- **C#**: 2000, Anders Hejlsberg presso Microsoft, evoluzione di C++ che ne rafforza il sistema di tipi e viene compilato nel Common Intermediate Language della Common Language Infrastructure, supportato il riuso di operatori ma non l'ereditarietà multipla, poi esteso con costrutti della programmazione funzionale.
- Esistono varianti a oggetti di linguaggi procedurali come Visual Basic per il Basic, Delphi per il Pascal e Objective-C per il C. Fortran, Cobol, Modula e Ada sono stati estesi in modo orientato agli oggetti.

1.3 Programmazione Dichiarativa: Funzionale e Logica

- Nel paradigma dichiarativo i programmi sono collezioni di espressioni matematico-logiche che descrivono cosa deve essere calcolato senza specificare come (anziché istruzioni di assegnamento da eseguire nell'ordine stabilito dalle istruzioni di controllo del flusso) e le loro esecuzioni sono assimilabili a deduzioni in un sistema formale (anziché sequenze di passi che modificano il contenuto della memoria). Questo comporta un notevole cambio di mentalità nell'attività di programmazione, come evidenziato da John Backus nel suo famoso articolo del 1978 intitolato "Can Programming Be Liberated from the Von Neumann Style? A Functional Style and Its Algebra of Programs".
- Principali differenze tra il paradigma dichiarativo e il paradigma imperativo:
 - I programmi dichiarativi sono espressi a un livello concettuale più alto dei programmi imperativi, così come i loro traduttori ed esecutori operano a un livello concettuale più alto rispetto a quelli convenzionali. Questo favorisce la verifica di correttezza dei programmi dichiarativi.
 - La nozione di stato della computazione intesa come il contenuto della memoria non esiste più nella programmazione dichiarativa perché i dati sono immutabili. In particolare, il valore di una variabile non cambia più una volta che è stato attribuito (legame anziché assegnamento).
 - In programmazione dichiarativa non ci sono effetti collaterali in quanto la valutazione di un'espressione non può modificare variabili al di fuori dell'ambiente locale. Questo è invece possibile in programmazione imperativa in presenza di variabili globali o parametri passati per indirizzo.
 - La conseguente trasparenza referenziale fa sì che venga restituito lo stesso risultato ogni volta che un'espressione viene valutata sugli stessi argomenti. Il risultato di una procedura che usa variabili globali o passa parametri per indirizzo potrebbe invece dipendere dal momento in cui è invocata.
 - Dal punto di vista prestazionale, il valore di un'espressione complessa su determinati argomenti può essere memorizzato in modo da non doverlo ricalcolare. Analogamente, due espressioni tra cui non ci sono dipendenze di dati possono essere valutate in qualsiasi ordine e persino in parallelo.
 - La ricorsione è praticamente l'unico strumento linguistico per rappresentare l'iterazione e il tipo di dato ricorsivo lista è praticamente l'unico tipo di dato strutturato disponibile. La gestione dinamica della memoria per le liste è completamente automatizzata (allocazioni e deallocazioni sono implicite e quindi non devono essere programmate come nel caso dei puntatori).

- La programmazione dichiarativa nasce in intelligenza artificiale per supportare il ragionamento automatico, la rappresentazione della conoscenza e l'elaborazione del linguaggio naturale. I programmi dichiarativi agevolano l'interazione con l'utente e il debugging da parte del programmatore perché la loro esecuzione consiste nell'applicazione di un ciclo leggi-valuta-stampa alle espressioni matematico-logiche presenti nei programmi. I linguaggi dichiarativi sono dunque interpretati, ma per motivi di efficienza vengono solitamente accompagnati anche da compilatori da utilizzare quando il codice diventa stabile.
- Il primo filone ad affermarsi è stato quello della programmazione funzionale. Esso si basa sul λ -calcolo sviluppato da Alonzo Church negli anni 1930, un sistema formale dello stesso potere computazionale della macchina di Turing universale per l'astrazione e l'applicazione di funzioni matematiche calcolabili. Un programma funzionale è pertanto una collezione di definizioni di funzioni e relative applicazioni e composizioni, che sfrutta la naturale trasparenza referenziale delle funzioni matematiche.
- Esempi di linguaggi funzionali (molti successivamente estesi con costrutti procedurali o a oggetti):
 - **Lisp**: 1958, John McCarthy presso MIT, lista come unica struttura dati i cui elementi sono racchiusi tra tonde e separati da spazi, funzioni espresse come liste in cui il primo elemento è il nome della funzione e i successivi sono gli argomenti, ricorsione e funzioni di ordine superiore, metaprogrammazione come capacità di trattare programmi come dati nonché di generare e modificare programmi, gestione dinamica della memoria comprensiva di garbage collection automatica, moltissimi dialetti.
 - **Scheme**: 1973, Steele e Sussman presso MIT, dialetto minimalista di Lisp, chiamate ricorsive ammesse solo in coda alla definizione di una funzione (maggiore efficienza perché non è necessario allocare un nuovo record sullo stack), strutturazione a blocchi per dichiarazioni di variabili, visibilità di una variabile statica solo all'interno della funzione in cui è dichiarata come in Algol e nei suoi successori anziché dinamica anche nelle funzioni invocate da quella funzione come in Lisp (maggiori chiarezza ed efficienza perché non è necessario scendere nello stack).
 - **ML**: 1973, Robin Milner presso Università di Edimburgo, linguaggio per il dimostratore automatico e interattivo di teoremi LCF, sintassi più simile a quella della matematica rispetto a Lisp, visibilità statica delle variabili, sistema di tipi comprensivo di inferenza automatica del tipo delle espressioni, pattern matching per gli argomenti delle funzioni, strutturazione a blocchi per la gestione delle eccezioni, moduli dotati di interfacce che dichiarano tipi e funzioni messi a disposizione (dialetti: SML presso Università di Edimburgo, Caml presso INRIA, F# presso Microsoft).
 - **Haskell**: 1990, comitato accademico europeo e americano, valutazione dei parametri effettuata solo nel momento in cui diventa necessaria (come in Miranda, progettato nel 1985 da David Turner in Inghilterra), visibilità statica delle variabili, sistema di tipi comprensivo di inferenza automatica del tipo delle espressioni, classi di tipi e polimorfismo, pattern matching per gli argomenti delle funzioni, principale implementazione Glasgow Haskell Compiler di Peyton-Jones e Marlow.
- Il secondo filone ad affermarsi è stato quello della programmazione logica. Esso si basa sulla logica dei predicati limitandosi a formule espresse come clausole di Horn per motivi di calcolabilità. Tali formule consentono di formalizzare sia fatti che rappresentano la conoscenza su un certo dominio, sia regole per ricavare ulteriore conoscenza applicando a tempo d'esecuzione il sistema di deduzione basato sulla risoluzione di Robinson guidata dalla strategia SLD di Kowalski. Un programma logico è pertanto una collezione di definizioni di predicati, considerati come relazioni matematiche che godono naturalmente di trasparenza referenziale, e relative applicazioni e composizioni che danno luogo a fatti e regole.
- Il principale linguaggio logico è il Prolog. Esso venne sviluppato nel 1972 da Colmerauer e Roussel presso l'Università di Marsiglia per l'elaborazione del linguaggio naturale, in collaborazione presso l'Università di Edimburgo con Kowalski, che fornì tra l'altro l'interpretazione procedurale delle clausole di Horn, e Warren, che poi implementò il primo compilatore. Le clausole di Horn che costituiscono fatti e regole in un programma Prolog seguono la sintassi dei predicati estesa col tipo di dato strutturato lista. La loro esecuzione è attivata tramite un'interrogazione espressa essa pure come clausola di Horn. Il risultato è un valore di verità accompagnato dai legami creati per le variabili eventualmente presenti nell'interrogazione. Prolog supporta sia la metaprogrammazione che il pattern matching per gli argomenti dei predicati e ha lo stesso potere computazionale della macchina di Turing universale.

1.4 Linguaggi di Programmazione Sequenziali e Concorrenti

- Nel paradigma imperativo un programma sequenziale è un programma in cui viene eseguita una sola istruzione alla volta sulla base del controllo del flusso stabilito dal programma stesso. Intendendo per stato della computazione il contenuto della memoria a un certo punto dell'esecuzione, il comportamento di un programma sequenziale viene formalizzato mediante una funzione matematica che descrive quale sia lo stato finale della computazione a fronte dello stato iniziale della computazione indotto da una generica istanza dei dati di ingresso, ignorando tutti gli stati intermedi della computazione.
- Un programma concorrente è invece composto da varie parti che possono essere eseguite contemporaneamente su un computer multiprocessore oppure su più computer collegati in rete. La sincronizzazione e la comunicazione tra le parti del programma avvengono mediante memoria condivisa nel primo caso e scambio di messaggi nel secondo caso. Il comportamento di un programma concorrente non può essere formalizzato tramite una funzione matematica che descrive l'effetto ingresso/uscita, in quanto il risultato calcolato dal programma può dipendere anche dalle velocità relative con cui vengono eseguite le parti del programma. Un modello più appropriato è un grafo stati-transizioni che riporta tutti i possibili stati finali a fronte di quello iniziale, dove ogni sequenza di transizioni è ottenuta interfogliando le istruzioni man mano eseguite dalle varie parti e mostra tutti gli stati intermedi attraversati.
- Tra i sistemi formali alla base della programmazione concorrente menzioniamo le algebre di processi, linguaggi algebrici aventi lo stesso potere computazionale della macchina di Turing universale. Esse sono dotate di operatori per rappresentare le composizioni sequenziale, alternativa e parallela di processi intesi come comportamenti. La loro semantica è basata su grafi stati-transizioni non deterministici dove la concorrenza è ridotta all'interfogliazione di computazioni locali. Come esempi citiamo CCS (1980, Milner), CSP (1984, Hoare), ACP (1984, Bergstra e Klop) e π -calcolo (1992, Milner). In altri formalismi come le reti di Petri (1962), il modello ad attori (1973, Hewitt) e le strutture di eventi (1980, Winskel), la concorrenza tra computazioni locali è invece rappresentata esplicitamente.
- Esempi di linguaggi concorrenti sono Occam (1983, David May presso Inmos, microprocessori, procedurale, CSP), Erlang (1986, Joe Armstrong presso Ericsson, telefonia, funzionale, CSP rivisitato con scambio di messaggi asincrono), Scala (2004, Martin Odersky presso EPF Losanna, a oggetti e funzionale, compatibilità con Java ma sintassi più compatta, reti di Petri). Anche linguaggi precedenti come C, Modula, Ada, C++, Java, C# e le varianti concorrenti di Pascal, Lisp, ML, Haskell, Prolog supportano in varia misura la programmazione concorrente.

1.5 Linguaggi di Script, Interrogazione, Markup, Modellazione

- Oltre ai linguaggi di programmazione e ai linguaggi visuali come Logo (1967, BBN) e Scratch (2003, MIT) pensati per avvicinare i più giovani, in informatica vengono impiegati pure altri linguaggi.
- I linguaggi di script vengono utilizzati per automatizzare l'esecuzione di compiti in contesti come sistemi operativi, server web e interfacce grafiche. Sono linguaggi interpretati dotati di una sintassi snella poi arricchitasi nel tempo, tra cui citiamo Unix shell (anni 1970), Perl/Raku (1987, Larry Wall, Unisys), Tcl/Tk (1988, John Ousterhout, UC Berkeley), Python (1991, Guido van Rossum, CWI), PHP (1995, Rasmus Lerdorf), JavaScript (1995, Brendan Eich, Netscape), Ruby (1995, Yukihiro Matsumoto).
- I linguaggi di interrogazione vengono utilizzati per consultare banche dati e modificarne i contenuti. Sono linguaggi interpretati dichiarativi il cui esempio più famoso è SQL (1974, Chamberlin e Boyce presso IBM, modello relazionale di Edgar Codd), seguito dai linguaggi per banche dati non relazionali.
- I linguaggi di markup vengono utilizzati per annotare documenti digitali con etichette sintatticamente distinguibili dal contenuto testuale che sortiscono il loro effetto nel momento in cui i documenti vengono visualizzati. Ne sono esempi HTML (1993, Tim Berners-Lee presso CERN, condivisione di documenti ipertestuali tramite Internet, pagine web multimediali interpretate dai browser), XML (1996, Jon Bosak presso W3C, formato comprensibile sia dalle persone che dalle macchine, estensibilità dell'insieme delle etichette), L^AT_EX (1984, Leslie Lamport presso SRI, preparazione di documenti scientifici, compilato).
- I linguaggi di modellazione vengono utilizzati per la progettazione software basata su modelli e abilitano la generazione e la verifica automatiche del software. Oltre ai linguaggi per la descrizione di architetture software, l'esempio più famoso è UML (1994, Rational Software, modello di sviluppo software a oggetti di Grady Booch, diagrammi del comportamento e della struttura dei sistemi software). ■ftplf_1